

Literature Review: TinyML-Based Smart Intrusion Detection System with ESP32

A Review of Current Research and Implementations

March 6, 2026

Abstract

The proliferation of Internet of Things (IoT) devices has enabled the development of affordable and intelligent security solutions for residential and commercial environments. The EagleEye Smart Intrusion System represents a convergence of several cutting-edge technologies: Tiny Machine Learning (TinyML), edge computing, embedded vision systems, and real-time communication protocols [1, 2]. This literature review examines the foundational research, current methodologies, and technological frameworks that underpin the development of an intrusion detection system utilizing the ESP32 microcontroller for local image processing, secure cloud storage, and real-time mobile notifications. The core objective of this project is to implement person detection algorithms directly on resource-constrained embedded devices, specifically the ESP32-CAM module, which can capture images of individuals at expected entry points and transmit alerts via messaging platforms without relying on continuous cloud-based processing. This approach addresses critical concerns regarding privacy, latency, cost-effectiveness, and energy efficiency that characterize traditional cloud-dependent surveillance systems [2].

1 Tiny Machine Learning (TinyML) Fundamentals

1.1 Definition and Scope

Tiny Machine Learning (TinyML) represents the intersection of machine learning, embedded systems, and IoT technology, enabling the deployment of neural network models on microcontroller units (MCUs) with severely constrained computational resources [4, 5]. TinyML systems typically operate within memory constraints of a few hundred kilobytes and power budgets measured in milliwatts, making them suitable for battery-powered, always-on applications [6, 7].

The TinyML paradigm has experienced remarkable growth since its formal introduction in 2019, with research publications exhibiting widespread acceleration in both academic research and commercial applications [8]. This growth is driven by advances in model compression techniques, specialized hardware accelerators, and optimized inference frameworks designed specifically for microcontrollers.

1.2 Key Enabling Technologies

Several technological advances have made TinyML feasible for computer vision applications. Neural networks trained on powerful hardware must undergo substantial opti-

mization before deployment on microcontrollers. Quantization techniques convert 32-bit floating-point weights to 8-bit integers (Int8), reducing model size by approximately 75% while maintaining acceptable accuracy levels [9, 10]. Research demonstrates that properly quantized models can achieve high accuracy retention compared to their full-precision counterparts [7, 11].

TensorFlow Lite for Microcontrollers (TFLite Micro) has emerged as the de facto framework for deploying machine learning models on embedded devices [11]. The framework has been successfully ported to multiple architectures including the ESP32. TFLite Micro operates without requiring an operating system or standard C/C++ libraries, making it highly portable [12]. Furthermore, the Edge Impulse platform provides end-to-end workflows for data collection, model training, and deployment. It offers compiling tools that optimize TensorFlow Lite models for specific microcontroller architectures like the ESP32 [13, 14].

1.3 Energy Efficiency Considerations

Energy efficiency represents a critical design constraint for battery-powered embedded vision systems. TinyML systems can achieve significant energy savings compared to traditional machine learning systems [6]. A comprehensive study on TinyML energy viability for computer vision applications using ESP32 revealed that camera-equipped prototypes consumed 1.25 volts per hour during active operation, while deep sleep configurations showed improved longevity at 0.93 volts per hour [6]. These findings emphasize the importance of intelligent power management strategies, including deep sleep modes and event-driven activation via PIR sensors, to maximize operational lifetime in deployment scenarios.

2 ESP32 Platform for Embedded Vision

2.1 Hardware Architecture

The ESP32 family of microcontrollers has emerged as a popular platform for TinyML applications due to its combination of processing capability, integrated wireless connectivity, and affordability [4]. The ESP32 features a dual-core 32-bit processor operating at up to 240 MHz and 520 KB of SRAM [4, 15]. The dual-core architecture enables parallel processing, allocating network communication to one core while the other handles inference tasks.

The ESP32-CAM variant integrates an OV2640 camera module capable of capturing 2-megapixel images, making it particularly suitable for vision-based applications [2, 16]. The camera supports multiple resolution modes; however, lower resolutions optimized for machine learning inference, such as 96×96 or 48×48 pixels, are preferred to preserve memory [2, 17].

2.2 Computational Capabilities and Limitations

While the ESP32 provides impressive capabilities, it faces significant constraints. Complex object detection models can overwhelm the ESP32's computational capacity, resulting in low frame rates (1-3 FPS) [18, 19, 20, 21]. Limited SRAM restricts the size

and complexity of deployable models, often leading to camera initialization failures or brownout detection problems if memory is not managed carefully [15, 22].

2.3 Successful Implementation Strategies

To overcome these limitations, research identifies several strategies. Resolution optimization, such as reducing input to grayscale 48×48 , significantly decreases computational requirements [17]. Lightweight architectures like MobileNet-SSD or custom shallow CNNs demonstrate optimal performance on these devices [23, 24]. While EagleEye emphasizes local processing, integrating it with a Python gateway ensures that heavy network protocols do not interrupt the camera’s primary inference loop.

3 Person Detection Algorithms for Embedded Systems

3.1 Dataset Requirements and Model Architectures

High-quality datasets are fundamental to training effective person detection models for TinyML. The Visual Wake Words (VWW) dataset has served as the standard for TinyML person detection, though newer datasets like Wake Vision provide broader coverage of scenarios [25, 26].

MobileNet has become a predominant architecture for embedded vision due to its depthwise separable convolutions [27, 28, 29]. However, standard MobileNet can still be heavy for an ESP32. Specialized architectures like FOMO (Faster Objects, More Objects) predict object center points rather than bounding boxes, reducing computational overhead substantially [23, 24]. Our implementation favors custom, shallow CNNs with heavy MaxPooling layers to drastically cut tensor operations, achieving latency as low as 112ms.

4 Edge Computing and Privacy Preservation

4.1 Edge Computing Architecture

Edge computing processes data locally at or near the source, contrasting with traditional cloud architectures that transmit raw data to remote servers [2, 30, 31]. For surveillance, edge computing provides latency reduction by eliminating round-trip communication delays [2]. Most importantly, it enhances privacy. Processing images locally without continuous cloud transmission protects sensitive visual information, ensuring recordings only leave the device upon confirmed detection [30, 31, 32].

4.2 Privacy-Preserving Techniques

Research on privacy-preserving edge computing demonstrates that maintaining data locally prevents direct data transmission between camera modules and external servers unless strictly necessary [33]. The EagleEye system acts as a privacy buffer. If no human is detected, images are instantly discarded. Only when the TinyML model triggers a detection event (accuracy $\geq 80\%$) is an image securely forwarded.

5 Communication Protocols and System Integration

5.1 Messaging APIs and MQTT

Earlier iterations of ESP32 connectivity relied heavily on third-party messaging integrations like the CallMeBot API for WhatsApp or Telegram [3, 34, 35, 36, 37]. While simple, these HTTP-based approaches impose rate limits and blocking tasks on the microcontroller.

A superior approach for integrating IoT devices is MQTT (Message Queuing Telemetry Transport). MQTT enables decoupled communication between the camera and advanced notification services through a message broker [38]. By publishing lightweight payloads (e.g., to the ‘eagleeye/camera/image’ topic), the ESP32 delegates heavy network operations.

5.2 Gateway and Cloud Architecture

Transmitting images poses technical challenges. High-resolution JPEGs require substantial memory. In advanced implementations like EagleEye, a local Python Gateway bridges the MQTT broker to robust cloud services. It validates image integrity (checking Magic Bytes like 0xFFD8) before securely uploading to Cloudinary [40]. Real-time synchronization is achieved by logging the secure URL to Firebase Realtime Database, which instantly triggers updates on a React Native mobile application [41].

6 Real-Time Detection and Response Systems

6.1 Sensor Fusion and Robustness

Effective real-time intrusion detection systems integrate multiple components. PIR motion sensors serve as low-power triggers that activate the camera and inference pipeline only when movement is detected, maximizing battery life [37]. Furthermore, integrating audio intelligence (acoustic threat detection for glass breaking or footsteps) provides a highly resilient, multi-modal sensor fusion approach [37, 42].

7 Challenges, Limitations, and Conclusions

The ESP32 platform imposes strict constraints. TinyML models must fit within 400 KB of flash and utilize less than 200 KB of RAM during inference, necessitating aggressive model compression [15, 18, 20]. Reducing model complexity inherently impacts detection accuracy across varied lighting and environmental conditions [39].

However, the convergence of TinyML, edge computing, and affordable embedded hardware creates unprecedented opportunities for intelligent, privacy-preserving security. The literature demonstrates that appropriately optimized neural networks can execute person detection on ESP32 hardware efficiently [2, 15, 25].

The EagleEye Smart Intrusion System represents a practical, advanced application of these research findings. By utilizing Int8 quantized models, grayscale optimization, MQTT protocols, and a decoupled Python gateway architecture, it achieves an inference latency of 112ms. This approach addresses the fundamental limitations of cloud-

dependent alternatives regarding bandwidth, privacy, and system reliability, representing a significant stride in decentralized smart surveillance.

References

- [1] S. Tanwar et al., “Privacy-Preserving Surveillance Using Edge Computing,” *2020 IEEE International Conference on Computing, Power and Communication Technologies*, 2020.
- [2] PMC, “Design and Implementation of ESP32-Based Edge Computing for Object Recognition,” 2025.
- [3] Random Nerd Tutorials, “ESP32: Send Messages to WhatsApp,” 2024.
- [4] arXiv, “From Tiny Machine Learning to Tiny Deep Learning: A Survey,” 2025.
- [5] ScienceDirect, “A review on TinyML: State-of-the-art and prospects.”
- [6] MedCrave, “Evaluation of the energy viability of smart IoT sensors using TinyML for computer vision applications,” 2023.
- [7] IEEE, “Real-Time Human Detection and Counting System Using Deep Learning Computer Vision Techniques,” 2022.
- [8] SIGARCH, “Tiny Machine Learning: The Future of ML is Tiny and Bright,” 2024.
- [9] arXiv, “Detecting Small Objects in Real-Time on Microcontroller Units,” 2024.
- [10] ScienceDirect, “Combinative model compression approach for enhancing 1D CNN performance.”
- [11] Fritz AI, “What is TensorFlow Lite Micro,” 2023.
- [12] YouTube, “Simple ESP32-CAM Object Detection,” 2023.
- [13] TinyWorld, “How to Deploy Model to Arduino,” 2021.
- [14] Edge Impulse, “Deployment Documentation,” 2025.
- [15] Semantic Scholar, “Camera Image Processing on ESP32 Microcontroller with Help of Convolutional Neural Network,” 2022.
- [16] ScienceDirect, “Multicamera edge-computing system for persons indoor location and tracking,” 2023.
- [17] Hackster.io, “Embedded Machine Learning for Person Detection,” 2022.
- [18] Reddit, “ESP32-CAM Project: Human Detection, Image Capture,” 2024.
- [19] Arduino Forum, “How to Perform Object Detection Directly on ESP32-CAM,” 2024.
- [20] YouTube, “ESP32 CAM with Python OpenCV Yolo V3 for object detection,” 2023.
- [21] Arduino Forum, “Person detection model on ESP32-WROVER,” 2022.

- [22] Random Nerd Tutorials, “ESP32-CAM Troubleshooting Guide,” 2024.
- [23] arXiv, “Detecting Small Objects in Real-Time on Microcontroller Units,” 2024.
- [24] Semantic Scholar, “TinyML-Based Pothole Detection: A Comparative Analysis of YOLO and FOMO.”
- [25] arXiv, “Wake Vision: A Tailored Dataset and Benchmark Suite for TinyML Person Detection,” 2024.
- [26] TensorFlow Blog, “Introducing Wake Vision: A New Dataset for Person Detection in TinyML,” 2024.
- [27] IJESAT, “Real Time Object Detection Using MobileNet-SSD,” 2024.
- [28] IJERT, “Real-time object detection and recognition using MobileNet-SSD with OpenCV,” 2024.
- [29] IEEE, “An Improved Object Detection Model for Embedded Systems,” 2020.
- [30] PMC, “Smart Video Surveillance System Based on Edge Computing,” 2021.
- [31] arXiv, “Efficient Privacy Preserving Edge Computing Framework for Image Classification,” 2020.
- [32] PMC, “Intelligent Monitoring System with Privacy Preservation Based on Edge AI,” 2023.
- [33] USENIX, “Privacy-Aware Deep Learning Approach for Face Recognition,” 2018.
- [34] Engineers Garage, “Send messages to WhatsApp or Telegram from ESP32,” 2024.
- [35] ElectronicWings, “Send a WhatsApp message using ESP32,” 2024.
- [36] TechTOnions, “ESP32 Sending Alert Message To WhatsApp,” 2025.
- [37] PMC, “Developing real-time IoT-based public safety alert and emergency response system,” 2025.
- [38] JETIR, “Smart Doorbell Security System Using IoT,” 2022.
- [39] PMC, “Research on Target Image Classification in Low-Light Night Vision,” 2024.
- [40] IJSRST, “Smart Home IoT Device Implementation,” 2021.
- [41] PMC, “Usage and impact of the internet-of-things-based smart home technology,” 2022.
- [42] Pioneer Security, “How Edge computing in surveillance Improves Security,” 2025.